GPU-First Heisenberg-Picture Tensor Network Dynamics for the 2D Transverse-Field Ising Model

Paolo D'Alberto*
Advanced Micro Devices, Inc.
paolo.dalberto@amd.com

May 15, 2026

Abstract

We present CppSim, a C++/HIP 2D Ising simulator for Heisenberg-picture tensor network time evolution on AMD GPUs. Using float32 throughout instead of the ComplexF32 representation of the Julia reference, CppSim halves the memory footprint per element, enabling $\chi_{\max} = 120$ on a single 32 GB GPU where the reference tops out at $\chi_{\max} \approx 75$, while achieving twice the throughput at all bond dimensions. The key contributions are: First, a zero-malloc GPU workspace that pre-allocates all buffers at startup; Second, a custom HIP tensor permutation kernel replacing host-side index shuffling with a pure device-to-device operation, yielding a 7.6× trotter speedup; Third, a hybrid QR strategy selecting Cholesky-QR for tall-skinny matrices and Householder-QR otherwise; Fourth, adaptive Belief Propagation with log-space Bethe partition function evaluation and explicit sign tracking. We validate correctness against golden-value tests and demonstrate χ-convergence of the spin autocorrelation function $C(t)$ for $\chi \in \{50, 70, 80, 100\}$ on a 4×4 grid. An analytic memory model, validated to within 2% against hardware measurements, projects feasibility for a 10×10 grid at $\chi_{\max} = 100$ on AMD MI350 hardware (≈ 109 GB HBM). Performance across AMD GPU generations (MI60 GFX906, MI100 GFX908) reveals that at saturation the workload is memory-bandwidth-bound, and that architecture-specific GEMM kernel paths affect both timing and numerical reproducibility at large χ.

1 Introduction

Simulating real-time quantum dynamics of strongly correlated many-body systems is a central challenge in computational physics. Tensor network (TN) methods provide a systematically improvable approximation by representing quantum operators as networks of low-rank tensors [1], with accuracy controlled by the bond dimension χ. While one-dimensional systems are well-served by Matrix Product State (MPS) methods [2], two-dimensional (2D) tensor networks are increasingly important for studying frustrated magnets, quantum circuits, and non-equilibrium dynamics at accessible system sizes.

The broader motivation for this work is the Multi-Product Formula (MPF) framework for hybrid quantum-classical computation. In MPF, a quantum computer executes Trotter circuits to approximate Hamiltonian time evolution, and a classical linear regression over multiple circuit runs at different step sizes cancels the leading Trotter error terms. The spin autocorrelation function

*Advanced Micro Devices, Inc. (AMD). paolo.dalberto@amd.com. This work was developed in active collaboration with Claude (Anthropic)

$C(t) = \mathrm{Tr}[Z_c(t)\, Z_c]/2^N$ is the quantitative figure of merit: it is what the ideal, error-free circuit would compute, and what the MPF post-processing corrects toward. CppSim provides the classical reference for this quantity.

The *Heisenberg picture* [9] is the natural language for this setting: the observable Z_c is evolved forward in time under the Hamiltonian while the infinite-temperature state remains fixed, which maps directly to how a quantum circuit acts on an operator. The correlation function is then extracted as an inner product with the initial operator — no state preparation or measurement of a full density matrix is required. Computing $C(t)$ exactly would require contracting the full 2D tensor network, an exponentially costly operation. Belief Propagation (BP) [10] makes this tractable: each lattice site iteratively sends its neighbors a message summarizing everything it sees from its side of the network; after convergence, each site holds an approximate picture of the global environment using only local information. The approximation rests on the Bethe assumption: on a tree, a site's dependence on the rest of the network passes entirely through its direct neighbors — there are no alternative paths — making BP exact.

Existing implementations of these methods are primarily in Julia [9], benefiting from high-level tensor abstractions and just-in-time compilation. However, Julia's garbage collector imposes non-trivial overhead for GPU workloads: allocation patterns during simulation trigger frequent collection cycles, and GPU memory pool management introduces latency spikes. For production-scale computations at large bond dimensions and grid sizes, these overheads become significant.

We present CppSim, a C++/HIP implementation that addresses these limitations through explicit GPU memory management, GPU-native linear algebra, and a zero-malloc simulation loop, eliminating the memory pool and its management overhead entirely. The implementation targets AMD GPUs through the HIP portability layer and the rocBLAS/rocSOLVER libraries [4, 5].

Contributions. The main contributions of this work are:

1. A **zero-malloc GPU workspace** design that pre-allocates all buffers at startup, making peak VRAM usage known exactly before the first layer runs and eliminating all allocator overhead during simulation.
2. A **custom HIP tensor permutation kernel** that replaces host-side index shuffling (requiring D2H+H2D transfers) with a pure device-to-device operation, yielding a 7.6× trotter speedup.
3. An **adaptive QR strategy** combining Cholesky-QR for tall-thin matrices at saturation with Householder-QR for small bond dimensions, with explicit numerical safeguards.
4. A **sign-correct log-space Bethe partition function** with explicit Bethe edge correction and per-bond sign tracking, resolving systematic C(t) anomalies at late times.

Because all buffer sizes are determined analytically at startup, the memory layout yields a closed-form model (`estimate_memory.py`) that predicts VRAM requirements for any grid and bond dimension, validated to within 2% against hardware measurements and used here for deployment planning across GPU generations.

The remainder of this paper is organized as follows. Section 2 describes the mathematical formulation. Section 3 presents the software design. Section 4 details the core algorithms. Section 5 describes the implementation. Section 6 presents numerical experiments and physics results. Section 7 reports performance. Section 8 discusses GPU portability. Section 9 concludes.

2 Mathematical Formulation

This section establishes the physical model and the mathematical framework underlying the simulation. We work entirely in the Heisenberg picture, where the observable rather than the quantum state is evolved in time, and we represent it as a tensor network on the lattice graph.

2.1 Model

We consider the 2D transverse-field Ising model [6] on a $k \times l$ rectangular lattice with open boundary conditions and Hamiltonian:

$$\mathcal{H} = -J \sum_{\langle ij \rangle} X_i X_j \; - \; h \sum_i Z_i, \tag{1}$$

where X_i, Z_i are Pauli operators on site i, J is the Ising coupling, h is the transverse field strength, and the sum is over nearest-neighbor pairs.

2.2 Heisenberg Picture and Spin Autocorrelation

The spin autocorrelation function measures how much the observable Z_c at the lattice center remembers its initial value after time t:

$$C(t) = \mathrm{Tr}\big[\rho_0 \, Z_c(t) \, Z_c\big], \tag{2}$$

where ρ_0 is the initial state. In the Heisenberg picture, the state ρ_0 remains fixed while the observable evolves under the Hamiltonian:

$$Z_c(t) = e^{i\mathcal{H}t} \, Z_c \, e^{-i\mathcal{H}t}. \tag{3}$$

Choosing ρ_0 constant (proportional to the identity), the trace simplifies to:

$$C(t) = \frac{1}{2^N} \mathrm{Tr}\big[Z_c(t) \, Z_c\big], \tag{4}$$

where 2^N is the Hilbert space dimension (N sites), which is the quantity computed throughout this work and targeted by the MPF post-processing described in Section 1.

2.3 Pauli Basis Tensor Network

Each qubit site has a 2-dimensional Hilbert space, but the space of 2×2 Hermitian operators has dimension $D = 4$, spanned by the Pauli matrices $\{I, X, Y, Z\}$ — orthogonal under the Hilbert-Schmidt inner product $\mathrm{Tr}(A^\dagger B)$. The operator $Z_c(t)$ is represented as a tensor network on the lattice graph: each site v carries a tensor ψ_v whose indices encode both the bond entanglement with neighboring sites and the local Pauli component of the operator.

Concretely, $\deg(v)$ denotes the number of nearest-neighbor bonds of site v: 2 at corners, 3 at edge sites, and 4 at interior sites on a rectangular grid. The tensor ψ_v is pre-allocated with one bond index of dimension $\chi_{\max}$ per neighbor and one Pauli index of dimension $D = 4$, giving a maximum shape of $\chi_{\max}^{\deg(v)} \times D$. The live bond dimension χ_{live} grows from 1 to $\chi_{\max}$ as entanglement builds across the first Trotter layers; all computations use the current live dimension, not the pre-allocated maximum. The entry $\psi_v[i_1, \ldots, i_{\deg(v)}, p]$ is the coefficient of the p-th Pauli matrix (p=0: I, p=1: X, p=2: Y, p=3: Z) at site v, for bond state $(i_1, \ldots, i_{\deg(v)})$. At $t = 0$: the center site has $\psi_c[\cdot, 3] = 1$ (pure Z); all others have $\psi_v[\cdot, 0] = 1$ (pure I).

The bond entanglement between neighboring sites a and b is captured by a $\chi \times \chi$ message matrix $\mu_{a\to b}$, one per directed bond, stored in the Vidal canonical gauge [3]. In this gauge, each $\mu_{a\to b}$ holds the Schmidt singular values of the bipartition of the network across bond (a, b), with the site tensors orthogonalized so that all entanglement information is encoded in the messages. Truncating each bond to rank $\chi_{\max}$ then discards only the least significant singular values — the locally optimal approximation at fixed bond dimension. On a tree this gauge is exact; on the 2D lattice BP finds the approximate Vidal gauge, which is why BP runs before every correlation function evaluation.

2.4 Trotter Decomposition

Each time step δt advances the operator by unitary conjugation:

$$Z_c(t + \delta t) = U^\dagger(\delta t)\; Z_c(t)\; U(\delta t), \qquad U(\delta t) = e^{-i\mathcal{H}\delta t}, \tag{5}$$

where $U^\dagger = e^{+i\mathcal{H}\delta t}$ acts on the left and U on the right. Because the Hamiltonian (1) contains only real Pauli couplings, this conjugation maps Pauli matrices to real linear combinations of Pauli matrices, so the gate acting on ψ_v in the Pauli basis is a real orthogonal matrix — no complex arithmetic is required.

The unitary $U(\delta t)$ is itself approximated to first order as a composition of simpler gates (the Trotter decomposition [7]):

$$U(\delta t) \approx R_z(h\,\delta t) \cdot R_{xx}(2J\,\delta t) \cdot R_z(h\,\delta t), \tag{6}$$

where R_z is a single-site rotation (diagonal in the Pauli basis) and R_{xx} is a two-site entangling gate applied to all nearest-neighbor pairs via a 4-color decomposition: bonds are partitioned into four color classes such that no two bonds in the same class share a site, enabling sequential independent updates.

2.5 Gate Update

For each two-site gate on bond (v_1, v_2), let $i \in \{1, 2\}$ index the two sites. The message matrices $\mu_{a\to b}$ (defined in §2) are diagonal in the Vidal gauge, holding singular values, so $\sqrt{\mu}$ denotes the diagonal matrix of their square roots. The update proceeds as follows (see Algorithm 1):

1. **Environment absorption.** Contract ψ_{v_i} with $\sqrt{\mu_{e\to v_i}}$ on every bond e incident to v_i except the active bond (v_1, v_2). This yields the *contraction vector* $\mathbf{cv}_i$ — so named because it is the site tensor contracted with its environment, ready for QR — which folds the environmental context into each site before the update.
2. **QR decomposition.** Factor each gauged tensor: $\mathbf{cv}_i = \mathbf{Q}_i\mathbf{R}_i$, where $\mathbf{Q}_i$ ($\chi_{\max}^3 \times D\chi_{\max}$) is isometric and $\mathbf{R}_i$ ($D\chi_{\max} \times D\chi_{\max}$) is compact. QR reduces the subsequent SVD from the intractable full tensor ($\chi_{\max}^3 \times D\chi_{\max}$) to a small bond matrix ($D\chi_{\max} \times D\chi_{\max}$).
3. **Bond matrix.** Form $\mathbf{RR} = \mathbf{R}_1^\top\mathbf{R}_2$ ($D\chi_{\max} \times D\chi_{\max}$), coupling the two sites at the bond interface.
4. **Gate contraction.** The gate $G \in \mathbb{R}^{D^2\times D^2}$ acts on the $D \times D$ physical (Pauli) block of $\mathbf{RR}$, leaving the bond indices unchanged:

$$\tilde{R}[d_1', d_2', k, l] = \sum_{d_1, d_2} G[d_1'd_2',\, d_1d_2]\; \mathbf{RR}[d_1, d_2, k, l].$$

5. **SVD and truncation.** Factor $\tilde{\mathbf{R}} = \mathbf{USV}^\top$, truncated to rank $\chi_{\max}$ (equation (11)).

6. **Reconstruction.** Set $\mathbf{cv}_i^{\text{new}} = \mathbf{Q}_i\mathbf{M}_i$, where $\mathbf{M}_1 = \mathbf{U}\sqrt{\mathbf{S}}$ and $\mathbf{M}_2 = \sqrt{\mathbf{S}}\mathbf{V}^\top$.
7. **Deabsorption.** Multiply by $(\sqrt{\mu_{e\to v_i}})^{-1}$ on all environment bonds to restore the Vidal gauge.
8. **Vidal normalization.** Store $\lambda_k \leftarrow s_k/\|\mathbf{s}\|_2$ ($\sum_k \lambda_k^2 = 1$), preventing float32 overflow at large χ.

2.6 Correlation Function via Belief Propagation

$C(t)$ (4) is the normalized Hilbert-Schmidt inner product of the evolved operator $Z_c(t)$ with the initial operator Z_c: $C(t) = \langle Z_c(t),\, Z_c\rangle_{HS}$, where $\langle A, B\rangle_{HS} = \mathrm{Tr}[A^\dagger B]/2^N$. Evaluating it requires contracting the full tensor network against the observable — a three-step process.

Step 1: trace to tensor network contraction. By Pauli orthogonality ($\mathrm{Tr}[P_aP_b] = 2\delta_{ab}$), the trace reduces to a single tensor network contraction: the coefficient of the configuration "Z at center c, I at all other sites" in the full network. This is computed by inserting the observable into the network and contracting.

Step 2: Bethe factorization. Contracting a 2D tensor network exactly is exponentially costly. After BP has converged the messages to the approximate Vidal gauge, the Bethe approximation (treating the lattice as a tree) factorizes the contraction as a product over sites divided by a product over edges:

$$C(t) = \frac{\prod_v vs_v}{\prod_e Z_e}, \tag{7}$$

where vs_v is the scalar contraction of ψ_v with its BP environment and the observable (Z at center c, I elsewhere), and

$$Z_e = \sum_i \mu_{a\to b}[i]\,\mu_{b\to a}[i] \tag{8}$$

is the Bethe edge correction: each bond appears in two site contractions, and Z_e removes the double-counting.

Step 3: log-space evaluation and sign tracking. The Z operator has eigenvalues ± 1, so the observable-specific BP messages can be negative, making vs_v and Z_e real-valued but not necessarily positive. Taking absolute values and separating signs before applying logarithms gives the numerically stable form:

$$C(t) = \exp\Big(\sum_v \log|vs_v| \;-\; \sum_e \log|Z_e|\Big) \times \underbrace{\prod_v \mathrm{sign}(vs_v)}_{\text{sign_num}} \times \underbrace{\prod_e \mathrm{sign}(Z_e)}_{\text{sign_den}}, \tag{9}$$

Omitting the sign factors would flip the sign of $C(t)$ whenever the product of signs across all sites and edges is -1 — a catastrophic error at late times when the sign structure of the BP messages becomes non-trivial (Fix I, Table 11).

3 Software Design

The overarching design principle is that GPU memory allocation is a one-time startup cost. Every buffer needed by the simulation — site tensors, BP messages, workspace intermediates — is allo-

cated at initialization at its maximum possible size and reused for every gate, layer, and Trotter step. This section describes the two main structures that realize this principle and the sizing rules that must be followed exactly.

3.1 Zero-Malloc Simulation Loop

The central design principle is that **no GPU memory is allocated or freed during simulation**. All buffers are allocated once at startup in two structures:

Workspace (grid-independent, reused for every gate):

Buffers	Size	Purpose
`cv1, cv2, Q1, Q2`	$\chi_{\max}^3 \times D\chi_{\max}$	gauged tensors, QR factors
`R1, R2, Rinv1, Rinv2, RR, U, Vt, gram, S`	$(D\chi_{\max})^2$ or $D\chi_{\max}$	QR/SVD matrices
`sqrt_env[4], inv_sqrt_env[4]`	$\chi_{\max}^2$	environment absorption
`svd_work`	$D^2\chi_{\max}$	GESVD bidiagonal scratch

IsingState (grid-dependent, persistent across all layers):

Buffers	Size	Purpose
`psi[`v`]`	$D \cdot \chi_{\max}^{\deg(v)}$	site tensors, pre-allocated at $\chi_{\max}$
`messages[bond][dir]`	$\chi_{\max}^2$	Vidal gauge, one per directed bond

Peak VRAM is reached at startup and remains constant throughout the run, eliminating GC pressure, fragmentation, and allocator contention.

3.2 Buffer Sizing and Safety Margins

The SVD bidiagonal scratch buffer (`svd_work`) requires careful sizing. The SVD is called on a matrix of dimensions $k_{1D} \times k_{2D}$ where $k_{iD} = k_i \times D$ and $k_i = \min(m_i, n_i)$. At bond dimension saturation, $m_i = \chi_{\max}^3 \gg n_i = \chi_{\max} D$, so $k_i = \chi_{\max} D$ and $k_{iD} = \chi_{\max} D^2$. The rocSOLVER `SGESVD` bidiagonal scratch (`E`) must therefore be sized to $\min(k_{1D}, k_{2D}) = \chi_{\max} D^2$ elements — a factor of $D = 4$ larger than the naive estimate $\chi_{\max} D$. The appendix (Fix K, Table 11) records the consequence of getting this wrong.

Both psi tensors and messages are allocated at their maximum possible size from startup, even when the actual bond dimension is much smaller at early layers. This trades $O(\chi_{\max}^4)$ memory at early layers for zero reallocation. The actual computation uses live `bond.chi` values, which grow from 1 to $\chi_{\max}$ over the first few Trotter layers.

4 Algorithms

CppSim was written entirely from scratch in C++/HIP, without reusing any code from the Julia reference. The physics, tensor operations, and BP were all re-implemented, and each required algorithmic choices that are specific to the GPU execution model and not present in the Julia implementation. Four components in particular represent original contributions: a GPU-native tensor permutation kernel that eliminates PCIe traffic entirely, an adaptive QR strategy exploiting

the tall-thin structure of the gauged contraction vectors, a truncated SVD on the compact bond matrix enabled by that QR, and a log-space Bethe partition function with explicit sign tracking that resolves systematic late-time errors. We describe each in turn.

4.1 GPU Tensor Permutation

Site tensors ψ_v are rank-5 objects (for interior sites) stored in canonical bond order [U, D, L, R, Pauli], where the Pauli index (dimension $D = 4$) selects one of the four matrices $\{I, X, Y, Z\}$ — each a 2×2 matrix with entries in $\{0, \pm 1\}$ (or $\pm i$ for Y).

QR decomposition operates on matrices (rank-2 tensors), so ψ_v must first be reshaped into one. The natural split is: group the *environment* bond indices (all bonds except the one being updated, dimension $\chi_{\max}^3$ for an interior site) into the rows, and group the *cut bond* together with the Pauli index (dimension $\chi_{\max} \times D = D\chi_{\max}$) into the columns. This produces a tall-thin matrix of shape $\chi_{\max}^3 \times D\chi_{\max}$, from which QR extracts an isometric $\mathbf{Q}$ (the environment) and a compact $\mathbf{R}$ (the bond interface). To enable this grouping, the cut-bond index must be moved adjacent to the Pauli index before reshaping — that is the sole purpose of the tensor permutation.

Permuting a rank-5 tensor of $\chi_{\max}^4 D$ elements on the CPU requires: a device-to-host (D2H) transfer ($\sim$100 MB per interior site at $\chi = 50$), an index-remapping loop, and a host-to-device (H2D) transfer back — totaling $\sim$9.6 GB of PCIe traffic per Trotter layer at $\chi = 50$.

Our custom HIP kernel eliminates all PCIe traffic. Each thread handles one output element by decomposing the input flat index (column-major) into a multi-index, applying the permutation, and computing the output flat index:

Listing 1: GPU tensor permutation kernel

```
__global__ void tensor_permute_kernel(
    const float* __restrict__ src,  // input tensor (column-major flat)
    float* __restrict__ dst,        // output tensor (permuted flat)
    PermuteMeta meta, int total)    // shape, strides, rank; passed by value
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x; // one thread per element
    if (idx >= total) return;

    int in_idx = idx, out_idx = 0;
    for (int d = 0; d < meta.ndim; d++) {
        int coord  = in_idx % meta.shape_in[d];   // coordinate along dim d
        in_idx    /= meta.shape_in[d];            // peel off dim d
        out_idx   += coord * meta.stride_out[d];  // accumulate permuted offset
    }
    dst[out_idx] = src[idx];  // scatter to permuted position
}
```

The `PermuteMeta` struct (shape, output strides, permutation, rank) is passed by value to avoid a device pointer indirection. This kernel reduced trotter time from $\sim$49,000 ms/layer to $\sim$6,400 ms/layer at $\chi = 50$ saturation — a **7.6× speedup** from this single change.

4.2 Adaptive QR Decomposition

At each gate application, a QR decomposition of the gauged tensor $\mathbf{cv} \in \mathbb{R}^{m \times n}$ is required, where $m = \chi_{\max}^3$ and $n = D\chi_{\max}$ at saturation. Two paths are used:

Cholesky-QR [8] (when $n > 150$ and $m/n > 150$; thresholds determined empirically on MI100 and left as tunable parameters, since the optimal crossover depends on both matrix dimensions and GPU architecture):

$$\mathbf{C} = \mathbf{cv}^\top \mathbf{cv} + \epsilon \mathbf{I}, \quad \mathbf{R} = \mathrm{chol}(\mathbf{C}), \quad \mathbf{Q} = \mathbf{cv}\,\mathbf{R}^{-1}, \tag{10}$$

with $\epsilon = 10^{-8}$ ensuring positive definiteness. This path uses rocBLAS `SGEMM` for $\mathbf{C}$ and `Q`, and rocSOLVER `SPOTRF` for the Cholesky factor. Since $\mathbf{R}$ is upper triangular, computing $\mathbf{R}^{-1}$ via host-side back-substitution is $O(n^2)$; however, it requires a D2H copy of $\mathbf{R}$, a GPU–CPU synchronization point (the GPU stalls while the CPU computes), and an H2D copy back before the final `SGEMM` for $\mathbf{Q} = \mathbf{cv}\,\mathbf{R}^{-1}$. The synchronization overhead, not the transfer size ($\leq 640\,\mathrm{KB}$), is the actual cost. A future optimization is to replace this with rocBLAS `STRSM`, which solves $\mathbf{Q}\,\mathbf{R} = \mathbf{cv}$ entirely on-device, eliminating the synchronization point.

Householder-QR (primary path at early layers; fallback at saturation):

$$(\mathbf{Q}, \mathbf{R}) = \mathrm{qr}(\mathbf{cv}),$$

via rocSOLVER `SGEQRF` + `SORGQR`. This path is backward stable and numerically safe regardless of the condition number of $\mathbf{cv}$, since it works directly on the matrix without forming the gram matrix. Both paths share the same D2H/H2D pattern — but for different reasons and at different cost. In the Householder path, `SGEQRF` stores the result in-place: the Householder reflectors occupy the lower triangle and $\mathbf{R}$ sits in the upper triangle of the same buffer. Extracting $\mathbf{R}$ separately requires a D2H copy of the full $m \times n$ buffer, host-side extraction of the upper triangle, and an H2D copy of $\mathbf{R}$ back — after which `SORGQR` generates $\mathbf{Q}$ on-device from the reflectors. At early layers ($n \leq 150$), χ is still small so the buffer is tiny and the round-trip is negligible. When invoked as a fallback at saturation, the same buffer is $\chi_{\max}^3 \times D\chi_{\max} \approx 1.6\,\mathrm{GB}$ at $\chi = 100$ — a transfer that makes Cholesky failure at large χ undesirable.[1] A future optimization is to extract $\mathbf{R}$ on-device via a triangular copy, eliminating the D2H entirely from both paths.

SVD and Truncation. The bond matrix $\mathbf{RR} \in \mathbb{R}^{k_{1D} \times k_{2D}}$ is factored via rocSOLVER `SGESVD`. The truncation rank is:

$$\chi_{\mathrm{new}} = \min\big(\#\{i : s_i \geq \epsilon_{\mathrm{cut}}\, s_0\},\ \chi_{\max}\big), \tag{11}$$

with default cutoff $\epsilon_{\mathrm{cut}} = 10^{-11}$. Singular values are Vidal-normalized before storage:

$$\lambda_i = s_i \,/\, \big(\textstyle\sum_j s_j^2\big)^{1/2}, \qquad \textstyle\sum_i \lambda_i^2 = 1. \tag{12}$$

By construction, $\sum_k \lambda_k^2 = 1$ implies $\lambda_k \in [0, 1]$ for any χ — float32 overflow of the message entries is impossible regardless of bond dimension. Without this normalization, overflow was observed in practice at $\chi \geq 50$ saturation.

BP Message Update. Each BP sweep updates all messages via:

$$\mu_{v \to e}^{\mathrm{new}}[i] \propto \psi_v[i, \ldots] \cdot \prod_{e' \neq e} \mu_{e' \to v}[i], \tag{13}$$

[1] In the Julia reference implementation, we instrumented the equivalent adaptive QR path and measured zero fallback invocations across all runs at all bond dimensions. Since the fallback would have been triggered by either a Cholesky failure or a numerical anomaly in the result, its absence confirms both: the Cholesky-QR path is stable in practice for this problem class, with no sign of numerical degradation — neither catastrophic nor mild.

implemented via rocBLAS strided-batched `SGEMV`. Two D2H transfers occur per message update: (1) *normalization*: χ scalars ($\leq$ 400 bytes) to compute the norm on the host for rescaling — unnecessary, since rocBLAS `snrm2` and `sscal` in `POINTER_MODE_DEVICE` could normalize entirely on-device (future optimization); (2) *convergence check*: χ^2 scalars ($\leq 40\,\mathrm{KB}$ at $\chi = 100$) to compare old and new message diagonals on the host — acceptable, since the host must make the stop/continue decision.

Adaptive convergence: the sweep loop exits when

$$\max_i |\mu^{\mathrm{new}}[i,i] - \mu^{\mathrm{old}}[i,i]| < \tau = 10^{-5}, \tag{14}$$

returning the sweep count. Most layers converge in 3–20 sweeps; hard layers (typically alternating layers at late times) require the full 100-sweep budget.

Algorithm 1 Gate update for bond (v_1, v_2)

Require: Site tensors ψ_{v_1}, ψ_{v_2}, gate G, messages μ, $\chi_{\max}$
Ensure: Updated ψ_{v_1}, ψ_{v_2}, updated message $\mu_{v_1 \leftrightarrow v_2}$

Compute $\sqrt{\mu}$, $(\sqrt{\mu})^{-1}$ via SYEVD for each env bond
$\mathbf{cv}_i \leftarrow \textsc{AbsorbEnv}(\psi_{v_i}, \sqrt{\mu}) \quad i = 1, 2$
$\mathbf{cv}_i \leftarrow \textsc{PermuteCutToLast}(\mathbf{cv}_i)$
$[\mathbf{Q}_i, \mathbf{R}_i] \leftarrow \textsc{AdaptiveQR}(\mathbf{cv}_i)$
$\mathbf{RR} \leftarrow \mathbf{R}_1^\top \mathbf{R}_2$ (via SGEMM)
$\tilde{\mathbf{R}} \leftarrow G(\mathbf{RR})$ (host loop over D^4 entries)
$[\mathbf{U}, \mathbf{s}, \mathbf{V}^\top] \leftarrow \textsc{Sgesvd}(\tilde{\mathbf{R}})$
$\chi_{\mathrm{new}} \leftarrow \min(\#\{s_i \geq \epsilon_{\mathrm{cut}} s_0\}, \chi_{\max})$
$\mathbf{cv}_i^{\mathrm{new}} \leftarrow \mathbf{Q}_i \mathbf{M}_i$ (bridge matrix $\mathbf{M}_i$ from $\mathbf{U}/\mathbf{V}^\top$, $\sqrt{\mathbf{s}}$)
$\mathbf{cv}_i^{\mathrm{new}} \leftarrow \textsc{InversePermuteCut}(\mathbf{cv}_i^{\mathrm{new}})$
$\mathbf{cv}_i^{\mathrm{new}} \leftarrow \textsc{DeabsorbEnv}(\mathbf{cv}_i^{\mathrm{new}}, (\sqrt{\mu})^{-1})$
$\psi_{v_i} \leftarrow \mathbf{cv}_i^{\mathrm{new}}$
$\mu_{v_1 \to v_2} \leftarrow \mu_{v_2 \to v_1} \leftarrow \mathrm{diag}(\lambda), \quad \lambda_i = s_i / \|\mathbf{s}\|_2$

5 Implementation

The simulator compiles as a single translation unit (`main.cpp`) with `hipcc`, with the Pauli dimension fixed at $D = 4$ at compile time; the only external dependencies are rocBLAS and rocSOLVER. All GPU-specific code lives in headers under `include/`, keeping the build a single invocation and enabling link-time optimization across the full gate pipeline.

Analytic Memory Model. The total GPU memory requirement for a $k \times l$ grid at bond dimension χ is:

$$\text{State:} \quad M_\psi = \left(4\chi^2 + 2(k+l-4)\chi^3 + (k-2)(l-2)\chi^4\right) \cdot D \cdot \texttt{sizeof(float)}, \tag{15}$$

$$M_\mu = 2\big(k(l-1) + (k-1)l\big)\,\chi^2 \cdot \texttt{sizeof(float)}, \tag{16}$$

$$\text{Workspace:} \quad M_W = \left(4\,\chi^3 \cdot D\chi + 9\,(D\chi)^2 + 8\,\chi^2\right) \cdot \texttt{sizeof(float)}. \tag{17}$$

The dominant terms are $M_\psi \sim (k-2)(l-2)\chi^4 \cdot D$ (interior psi tensors) and $M_W \sim 4D\chi^4$ (large workspace buffers). For a 4×4 grid both terms are comparable at $\chi \geq 50$.

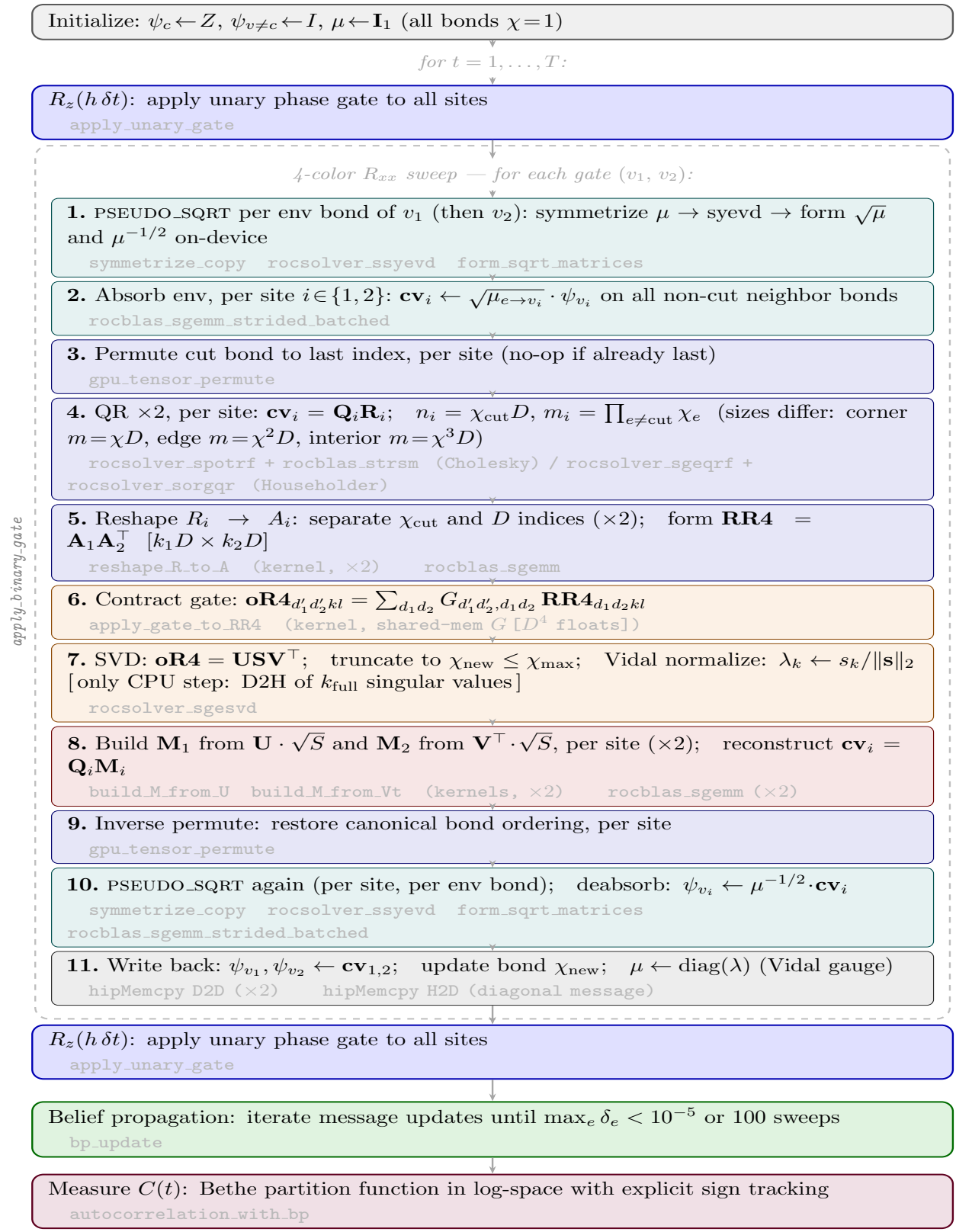


Figure 1: **CppSim computational pipeline for one time step.** Each box names the mathematical operation and the GPU primitive that executes it. The inner group (dashed) is `apply_binary_gate`, called once per bond per color class (≤ 24 times on a 4×4 grid). Color code: teal = environment conditioning; blue = linear algebra (QR/permute); orange = gate contraction and SVD; red = tensor reconstruction; green = belief propagation; purple = observable measurement. The only CPU computation in the hot path is the SVD truncation threshold (step 7), requiring a $D\chi$-float D2H transfer; all other steps execute on-device.

Table 1: Source file organization

File	Role
`main.cpp`	Driver, argument parsing, timing output table
`estimate_memory.py`	GPU memory predictor and FLOP estimator
`include/types.hpp`	`Scalar=float`, `HIP_CHECK`, `D=4`
`include/grid.hpp`	Lattice graph, site/bond topology, 4-coloring
`include/tensor.hpp`	`IsingState`: pre-allocated ψ and messages
`include/workspace.hpp`	GPU buffer pool, zero malloc during simulation
`include/permute_gpu.hpp`	HIP permutation kernel (§4.1)
`include/gate.hpp`	`apply_binary_gate`: full gate pipeline
`include/physics.hpp`	`IsingSimulator`, Trotter layer, R_{xx} layer
`include/measures.hpp`	BP update, autocorrelation, Bethe partition fn

6 Numerical Experiments

All experiments use $h = J = -1.0$, $\delta t = 0.1$, $\epsilon_{\text{cut}} = 10^{-11}$, and 20 Trotter layers ($t = 0$ to $t = 2.0$) in float32 precision. Hardware is AMD MI100 (GFX908) or MI60 (GFX906); both carry 32 GB HBM2 and differ only in compute architecture. Results use MI100 unless MI60 is noted.

6.1 Correctness Validation

The implementation was validated against:

1. **Julia reference**: $C(t)$ for 4×4 at $\chi = 50$ matches the Julia implementation to within float32 rounding ($< 10^{-6}$ absolute error) for $t \leq 16$.
2. **Chi consistency**: $C(t)$ for $t \leq 14$ agrees across $\chi \in \{50, 70, 80, 100\}$ to 5 significant figures, confirming that truncation error is negligible for early-to-mid time evolution.

6.2 Chi Convergence of $C(t)$

Table 2 shows the key findings:

- For $t \leq 14$, all four χ values agree to 5 significant figures.
- $\chi = 50$ gives $C(20) = 0.007225$, a well-known truncation artifact. $\chi \in \{70, 80, 100\}$ give $\{0.054961, 0.054254, 0.053758\}$, converging to ≈ 0.053.
- The residual gap between $\chi = 80$ and $\chi = 100$ at $t = 20$ is 0.0005 ($< 1\%$), indicating near-convergence.
- A minimum bond dimension of $\chi \approx 70$ is required for accurate late-time dynamics in this parameter regime.

6.3 Observations at Large Bond Dimension

The results at $\chi \in \{80, 100, 120\}$ are, to our knowledge, the first time this simulation has been run at these bond dimensions on a 4×4 grid. The Julia reference is memory-limited to $\chi \approx 75$ on 32 GB hardware; CppSim's float32 representation opens the range above that. The natural expectation is simple: larger χ means smaller truncation error and better physics. That expectation is confirmed

Table 2: Spin autocorrelation $C(t)$ on 4×4 grid (MI100) for $\chi \in \{50, 70, 80, 100\}$, with Julia float64 reference at $\chi = 50$. Agreement across all columns at early times confirms correctness; divergence at late times reveals truncation error.

t	$\chi = 50$	$\chi = 70$	$\chi = 80$	$\chi = 100$	Julia f64 $\chi = 50$
1	0.922619	0.922619	0.922619	0.922619	0.922619
2	0.724904	0.724904	0.724904	0.724904	0.724903
3	0.488101	0.488101	0.488101	0.488101	0.488101
4	0.291804	0.291804	0.291804	0.291804	0.291803
5	0.175283	0.175283	0.175284	0.175284	0.175284
6	0.131518	0.131518	0.131518	0.131518	0.131518
7	0.128375	0.128375	0.128375	0.128376	0.128375
8	0.134858	0.134858	0.134859	0.134859	0.134858
9	0.135000	0.135000	0.135000	0.134999	0.134999
10	0.126956	0.126956	0.126956	0.126955	0.126954
11	0.115129	0.115129	0.115131	0.115130	0.115127
12	0.103551	0.103551	0.103555	0.103555	0.103548
13	0.093672	0.093672	0.093682	0.093681	0.093667
14	0.085239	0.085239	0.085262	0.085257	0.085233
15	0.077760	0.077806	0.077793	0.077773	0.077752
16	0.071168	0.071198	0.071167	0.071107	0.071157
17	0.065696	0.065609	0.065524	0.065395	0.065682
18	0.061462	0.061130	0.060938	0.060710	0.061448
19	0.058280	0.057659	0.057278	0.009078*	0.058271
20	0.007225†	0.054961	0.054254	0.053758	0.055810

† Truncation artifact at $\chi = 50$ (CppSim float32).
* BP convergence anomaly (see §6.4).

for early and mid times — all χ values agree to 5 significant figures for $t \leq 14$. At late times and large χ, however, new phenomena appear that have no analog at smaller bond dimensions: BP sweep counts that spike by 50×, a transient sign anomaly at a single layer, and a smooth monotonic precision drift that separates GPU architectures. A contributing factor is that larger χ means larger internal reductions: the gram matrix $\mathbf{C} = \mathbf{c}\mathbf{v}^\top\mathbf{c}\mathbf{v}$ accumulates float32 rounding over χ^3 terms, and the Bethe partition function sums over more bonds and sites — making the result more sensitive to small errors precisely when $C(t)$ itself is small ($C(t) \lesssim 0.05$ at late times). We describe each phenomenon in turn.

A natural direction for future work is to run the same 4×4, $\chi = 100$ problem in float64 on a GPU with sufficient memory (e.g., AMD MI300X at $\approx 192\,\text{GB}$ HBM). Since the workload is memory-bandwidth-bound at saturation, the timing penalty of doubling the data width would be modest, while the precision gain would remove architecture-specific rounding as a variable — isolating whether the observed anomalies are numerical artifacts or genuine features of the physics at large χ.

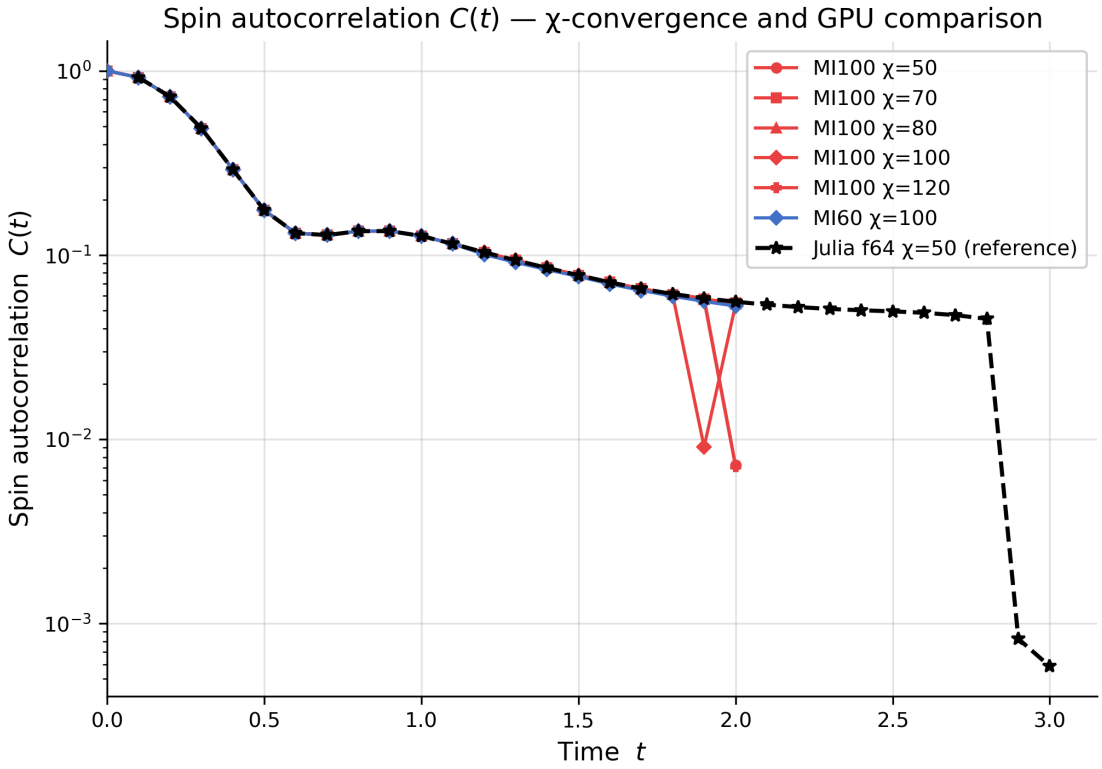


Figure 2: Spin autocorrelation $C(t)$ on a 4×4 grid (log scale) for $\chi \in \{50, 70, 80, 100, 120\}$ on MI100 (red), $\chi = 100$ on MI60 (blue), and the Julia float64 reference at $\chi = 50$ (black dashed, 30 layers). All curves overlap to within float32 rounding for $t \leq 1.5$, confirming correctness across GPU generations. The log scale reveals two distinct dynamical regimes separated by a kink at $t \approx 0.5$: a fast initial decay driven by nearest-neighbor gate spreading, followed by a slower plateau as entanglement saturates the bond dimension. Truncation artifacts appear as sharp downward spikes: $\chi = 50$ at $t = 2.0$ and a BP convergence anomaly at $\chi = 100$, $t = 1.9$ (see §6.4). The Julia reference exhibits its own truncation at $t \approx 2.9$ — the same physical mechanism at the same $\chi = 50$, delayed by float64 precision.

6.4 BP Fixed-Point Multiplicity and Damped Updates

Table 3 extends this comparison to $\chi \in \{50, \ldots, 100\}$ and two damping factors. Bold entries are metastable traps ($C(t) \approx 0.007$–0.010); NCVG entries did not converge within 100 sweeps. The pattern calls for explanation.

Two failure modes. Across the full (χ, t) parameter space we observe two qualitatively distinct BP failure modes that must not be confused. *Type 1 — metastable trap*: BP converges in a finite number of sweeps (sweep count elevated but delta $< \epsilon_{\text{tol}} = 10^{-5}$), yet the resulting $C(t)$ collapses to a characteristic value ≈ 0.007–0.010 regardless of χ, well below the physically expected trajectory. The sweep count at the trapped layer is 2–3× the typical value (e.g., 39 sweeps at $\chi = 80$, $t = 17$ vs. 12–18 on neighboring layers), providing an early-warning signal. *Type 2 — non-convergence*: BP exhausts the 100-sweep budget with delta $\gg \epsilon_{\text{tol}}$. The resulting $C(t)$ is unreliable but may still be physically reasonable; the next Trotter layer typically recovers. The sweep count alone does not

Table 3: Trap map: $C(t)$ at critical layers for $\chi \in \{50, \ldots, 100\}$ on MI100 and MI60 (undamped), and MI100 with damped BP ($\alpha \in \{0.50, 0.80\}$). **Bold**: trapped value ($C(t) \approx 0.007$–0.010). NCVG: non-convergent (100 sweeps, delta $> \epsilon_{\text{tol}}$). The reference value at $t = 17$ is ≈ 0.066 (MI60 undamped $\chi = 80$; MI100 damped $\alpha = 0.50$, $\chi = 80$: 0.066).

		$\chi_{\max}$					
Configuration	Layer	50	60	70	80	90	100
MI100 undamped	$t = 17$	0.069	0.067	**0.009**	**0.009**	0.065	0.064
	$t = 19$	0.058	0.058	**0.009**	0.057	**0.009**	0.057
MI60 undamped	$t = 17$	0.069	**0.027**	0.067	0.066	0.065	0.064
	$t = 19$	NCVG	**0.039**	NCVG	0.058	**0.009**	0.057
MI100 $\alpha = 0.50$	$t = 17$	0.069	0.067	0.066	0.066	0.065	0.064
	$t = 19$	**0.009**	0.058	0.058	NCVG	NCVG	NCVG
MI100 $\alpha = 0.80$	$t = 17$	0.069	0.067	0.066	**0.009**	0.065	0.064
	$t = 19$	**0.009**	0.058	0.058	0.057	NCVG	NCVG

distinguish the types; the final delta is essential.

Fixed-point multiplicity. The Bethe free energy on a loopy graph generically has multiple local minima. BP finds one of them depending on the trajectory through message space, which is determined by the update rule and the floating-point arithmetic path. Two controlled experiments provide direct evidence that the BP landscape has competing fixed points.

Cross-architecture experiment. The trap patterns in Table 3 are complementary across architectures: MI100 undamped traps at $(\chi = 80, t = 17)$ while MI60 undamped is clean; conversely, MI60 traps catastrophically at $(\chi = 60, t = 6)$ ($C(t) = 0.027$ vs. expected ≈ 0.067) while MI100 is clean. Both GPUs execute the same f32 arithmetic with the same algorithm, but MFMA and scalar-ALU reduction trees differ in rounding order, perturbing the BP trajectory enough to select a different basin. When both GPUs are un-trapped, $C(t)$ agrees to within 1–2×10^{-3}.

Update-rule experiment. Fixing $\chi = 80$ on MI100, we compare undamped BP ($\alpha = 1$), damped BP with $\alpha = 0.50$, and damped BP with $\alpha = 0.80$. Damping introduces momentum into the message update:

$$\mu_e^{(k+1)} \leftarrow (1 - \alpha)\, \mu_e^{(k)} + \alpha\, \hat{\mu}_e^{(k+1)}, \tag{18}$$

where $\hat{\mu}_e^{(k+1)}$ is the undamped update and $\alpha \in (0, 1]$. Spectrally, damping maps an eigenvalue λ of the BP Jacobian to $(1 - \alpha) + \alpha\lambda$, compressing the spectrum toward 1 and suppressing oscillatory modes that otherwise overshoot the target fixed point. At $(\chi = 80, t = 17)$: $\alpha = 0.50$ avoids the trap entirely ($C(t) = 0.066$), while $\alpha = 0.80$ and undamped both converge to the wrong fixed point ($C(t) = 0.009$, identical to six decimal places). At $(\chi = 80, t = 19)$: the roles reverse — $\alpha = 0.80$ finds the correct fixed point ($C(t) = 0.057$) while $\alpha = 0.50$ does not converge.

Convergence precision does not determine basin selection. A tighter convergence tolerance or more sweeps does not rescue a trapped run. Once BP has converged, it has found a valid fixed point of the Bethe equations; additional iterations confirm the wrong answer more precisely. What determines which basin is reached is the update rule (damped vs. undamped, choice of α) and the hardware floating-point trajectory — not the convergence threshold. This is confirmed by

the sweep-count data: the trapped ($\chi = 80, t = 17$) undamped run converges in 39 sweeps with delta $= 6.65 \times 10^{-6}$ (well below tolerance), while the un-trapped $\alpha = 0.50$ run at the same layer converges in 59 sweeps with delta $= 1.43 \times 10^{-6}$ (tighter). Tighter convergence, wrong answer.

Capacity boundary. A distinct third regime appears at high entanglement saturation ($S_e/\log\chi \gtrsim 0.50$): *both* α values fail simultaneously, either through non-convergence or entering the same wrong basin. At ($\chi = 90, t = 19$) and ($\chi = 100, t = 20$), the bond dimension itself is insufficient — no choice of α can compensate for an inadequate variational manifold. This boundary is cleanly separated from the fixed-point multiplicity regime: below it, at least one α recovers the correct $C(t)$; above it, none does.

Implications. The cross-architecture and cross-α experiments together establish two facts. First, BP finds different fixed points depending on the update rule and hardware, and these differences corrupt $C(t)$ when the wrong basin is selected. Second, the correct $C(t)$ can be recovered by choosing a different α — MI100 damped $\alpha = 0.50$ at ($\chi = 80, t = 17$) recovers $C(t) = 0.066$, matching MI60 undamped at the same configuration ($C(t) = 0.066$) to within 1.5×10^{-3}, confirming they reached the same physical fixed point by different paths. A future adaptive strategy should use the sweep count and entanglement saturation $S_e/\log\chi$ as posterior diagnostics: an elevated sweep count ($\gtrsim 40$) combined with $C(t)$ inconsistent with neighboring layers signals a Type 1 trap and warrants a retry with a different α. When $S_e/\log\chi \gtrsim 0.50$, the appropriate response is to increase χ rather than change the update rule.

6.5 Grid Scaling

Gate cost is independent of grid size (depends only on χ and site degree). Trotter time per layer scales linearly with n_{bonds}, as confirmed in Table 4: the ratio to 3×3 closely tracks $n_{\text{bonds}}/12$, with at most 10% deviation attributable to cache and memory-bandwidth effects at larger grids.

6.6 Memory Model Validation

The analytic model of Section 5 predicts peak VRAM from $\chi_{\max}$ and the grid dimensions alone, before the simulation runs. Table 6 compares predictions against `rocm-smi` measurements taken at the start of each run on a 4×4 grid. The model is accurate to within 2% for $\chi_{\max} \leq 100$, confirming that the dominant terms in equations (15)–(17) correctly account for the pre-allocated psi tensors, messages, and workspace. The 6% discrepancy at $\chi_{\max} = 120$ reflects an early measurement taken before all bond dimensions had grown to $\chi_{\max}$; the true peak would be closer to the prediction once all bonds saturate. This accuracy is sufficient for deployment planning: given a target grid and $\chi_{\max}$, the model determines whether a GPU can run the simulation before any code is compiled or executed. Table 7 extends the predictions to larger grids and bond dimensions, identifying the GPU generation required for each configuration.

7 Performance

We characterize performance along three axes: how Trotter time scales with bond dimension χ (empirical vs. theoretical), how CppSim compares to the Julia reference at equal χ, and how time is distributed across the three main operations (Trotter, BP, and correlation function evaluation) at saturation.

The trotter speedup of 1.5× comes primarily from the GPU tensor permutation kernel (7.6× improvement over CPU permutation, §4.1) and GPU strided-batched GEMM for environment absorption.

8 Portability

CppSim targets AMD GPUs through the HIP portability layer, but performance portability across GPU generations requires understanding architecture-specific behavior. This section compares MI100 and MI60 in both absolute timing and compute efficiency, and examines the Tensile kernel coverage gap and numerical precision differences that arise from distinct GPU compute architectures.

Both MI100 (GFX908, CDNA1) and MI60 (GFX906, Vega20) have 32 GB HBM2 and differ primarily in compute architecture (Table 9). At $\chi = 100$, 4×4, MI100 completes 20 Trotter layers in ≈ 65 min; MI60 averages ≈ 290 min, with individual saturated layers ranging from 155,000 ms to 1,430,000 ms due to Tensile kernel coverage gaps. BP cost doubles on MI60 ($\sim$100,000 ms vs. $\sim$50,000 ms on MI100), consistent with the 6.3× compute gap.

In Figure 4 and at full saturation ($\chi = 100$, layers ≥ 6) both settle below 1% of their respective peaks, confirming that the workload is *memory-bandwidth-bound* rather than compute-bound. The arithmetic intensity of the dominant Cholesky-QR and SVD operations falls between the MI60 roofline ridge point ($\approx$ 29 FLOP/byte) and the MI100 ridge point ($\approx$ 150 FLOP/byte); in this regime MI60 achieves a higher efficiency ratio relative to its peak while MI100 delivers greater absolute throughput ($\approx 1.2\times$ faster in wall-clock time despite a 6.3× higher compute ceiling). The large layer-to-layer variance on MI60 (blue) is a direct signature of Tensile kernel coverage gaps: layers that hit a pre-built kernel path achieve up to 5× higher efficiency, while fallback layers drop to MI100 levels. At $\chi = 120$, GFX906 lacks pre-built kernels for the arising GEMM shapes entirely, making $\chi \approx 110$ the practical ceiling on MI60; a GFX906-specific ROCm Docker image is required (GFX908 uses a separate image).

Numerical precision across architectures. A subtler portability issue is observed in the χ-convergence of $C(t)$: at $\chi_{\max} = 100$, MI60 produces values that drift systematically below MI100 by $1\text{–}3 \times 10^{-3}$ at late times ($t \geq 12$), with the deviation growing monotonically across layers. This drift is absent at $\chi_{\max} \leq 80$ and is not caused by sign anomalies in the Bethe partition function (which appear at all χ values on both GPUs and recover within one layer). The most likely source is the gram matrix GEMM in the Cholesky-QR path (§4.2): $\mathbf{C} = \mathbf{cv}^\top \mathbf{cv}$ is a float32 reduction over $k = \chi_{\max}^3$ terms ($k = 10^6$ at $\chi = 100$). GFX906 (Vega20) implements SGEMM via scalar vector-ALU instructions (no native matrix multiply units), while GFX908 (CDNA1) uses dedicated MFMA instructions with a different reduction tree. Floating-point addition is non-associative, so the two reduction orders yield different rounding errors; the effect grows with $k = \chi^3$, consistent with the observed χ-dependence of the drift. The Julia reference uses the same Cholesky-QR algorithm but in float64 (`_chol` variant), which makes the gram matrix computation essentially exact regardless of summation order.

Two experiments would isolate this mechanism. First, replacing the float32 gram matrix GEMM with a mixed-precision call (`rocblas_gemm_ex` with `compute_type = f64`) while keeping all other operations in float32 would confirm whether the gram matrix is the source: if MI60 and MI100 agree afterward, the reduction order in that single GEMM is responsible. Second, a complete float64 implementation at $\chi_{\max} = 50$ would provide an architecture-independent reference and enable direct

comparison with the Julia baseline. Since the workload at saturation is memory-bandwidth-bound (Section 7, Fig. 4), moving to float64 doubles the data traffic but does not increase the compute bottleneck; on both MI60 and MI100 the timing penalty would be modest and the accuracy gain substantial.

9 Conclusion

We have presented CppSim, a C++/HIP implementation of Heisenberg-picture tensor network dynamics for the 2D Ising model on AMD GPU hardware. Table 10 summarises the result against the Julia reference at $\chi = 50$, 4×4.

Table 10: Julia vs. CppSim at $\chi = 50$, 4×4.

	GC	Speedup	Memory	Mean $\lvert\Delta C(t)\rvert$ vs f64
Julia[†]	✓	1×	2× (ComplexF32)	$\sim 9 \times 10^{-4}$
CppSim	×	∼2.3×	1× (real float32)	$\sim 1.4 \times 10^{-5}$

[†]Index permutation runs on CPU; full GPU offload is future work.

CppSim runs bond dimensions up to $\chi = 120$ on a single 32 GB AMD GPU, a scale not previously reported for this model. The validated memory model projects the primary research target — 10×10 at $\chi = 100$, requiring ≈ 109 GB — within reach of AMD MI250X and MI350 hardware, opening a new scale of 2D quantum dynamics simulation.

Outlook. Beyond the immediate results, this work demonstrates that tensor network simulation at large bond dimension is a memory-bandwidth problem, not a compute problem: efficiency gains come from HBM bandwidth rather than FLOP count; real-number representation in the Pauli basis halves VRAM over complex storage; and on-device computation eliminates PCIe as the remaining bottleneck. Float32 precision itself becomes architecture-dependent at large χ, pointing toward mixed-precision accumulation as a necessary step for reproducible results across GPU generations. These are not CppSim-specific conclusions — they are design principles at the intersection where physicists choose what to simulate and computer scientists choose how.

Availability. Source code, build instructions, and experiment scripts are available at [repository URL].

References

[1] R. Orús. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. *Annals of Physics*, 349:117–158, 2014.

[2] G. Vidal. Efficient classical simulation of slightly entangled quantum computations. *Physical Review Letters*, 91(14):147902, 2003.

[3] G. Vidal. Efficient simulation of one-dimensional quantum many-body systems. *Physical Review Letters*, 93(4):040502, 2004. doi:10.1103/PhysRevLett.93.040502.

[4] AMD ROCm. rocblas: AMD ROCm BLAS library. https://github.com/ROCmSoftwarePlatform/rocBLAS, 2024.

[5] AMD ROCm. rocsolver: AMD ROCm LAPACK-like library. https://github.com/ROCmSoftwarePlatform/rocSOLVER, 2024.

[6] S. Sachdev. *Quantum Phase Transitions*. Cambridge University Press, Cambridge, 2nd edition, 2011. doi:10.1017/CBO9780511973765.

[7] M. Suzuki. Generalized Trotter's formula and systematic approximants of exponential operators. *Communications in Mathematical Physics*, 51(2):183–190, 1976.

[8] G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013. ISBN 9781421407944.

[9] M. S. Rudolph and J. Tindall. Simulating and sampling from quantum circuits with 2D tensor networks. *arXiv preprint arXiv:2507.11424*, 2025. https://arxiv.org/abs/2507.11424.

[10] J. Tindall and M. Fishman. Gauging tensor networks with belief propagation. *SciPost Physics*, 15:222, 2023. doi:10.21468/SciPostPhys.15.6.222.

A Correctness Fix History

During development, eleven correctness and performance issues were identified. Table 11 summarizes the most significant.

Table 11: Correctness and performance fixes during development.

Fix	File	Description
A	`gate.hpp`	Gate column index: `d1*D+d2` not `d1+D*d2`
B/C	`gate.hpp`	sv1/sv2 absorb must not interleave (shared workspace slots)
D	`main.cpp`	Center site: 0-indexed `site_id(nx/2-1, ny/2-1)`
E	`measures.hpp`	Observable BP: $d = 3$ (Z) at center, $d = 0$ (I) elsewhere
F	`measures.hpp`	BP on a *copy* of messages; in-place corrupts Vidal gauge
G	`measures.hpp`	Bethe edge: $Z_e = \sum_i \mu_{a\to b}[i]\,\mu_{b\to a}[i]$
H	`gate.hpp`	Vidal normalization $\sum \lambda^2 = 1$ prevents float32 overflow
I	`measures.hpp`	Explicit Z_e sign tracking in Bethe partition function
J	`measures.hpp`	Adaptive BP early exit (tol $= 10^{-5}$)
K	`workspace.hpp`	`svd_work` sized $D^2\chi_{\max}$ not $D\chi_{\max}$ ($4\times$ undersize)

Fix K: the `svd_work` buffer was sized at $\chi_{\max}D$ — a factor of $D = 4$ too small. The correct requirement is $\chi_{\max}D^2$ (derived in §5). This caused GPU page faults at $\chi_{\max} \geq 75$. The fault was invisible at $\chi_{\max} = 70$ because that run never reached chi saturation in 20 layers, keeping $k_{1D} < \chi_{\max}D^2$ and the overflow landing in already-mapped GPU memory.

B Memory Estimator

`estimate_memory.py` implements the analytic model of §5 and generates four tables:

Table 1 Total GPU memory (GB) for $k \times l$ grids ($k, l \in [3, 12]$) at $\chi \in \{10, 20, 30, 50, 70, 100, 120\}$. Entries exceeding 32 GB are flagged with `***`.

Table 2 Breakdown (psi/messages/workspace/measures) at $\chi = 50$ and $\chi = 100$, with maximum χ for 32 GB and 80 GB targets.

Table 3 Maximum $\chi_{\max}$ for MI60, MI100, MI250X, MI300X, A100 (40/80 GB), H100 across all grids.

Table 4 Per-gate FLOP estimates (QR + SVD + reconstruction GEMM) and per-layer TFLOP totals for representative configurations.

Table 4: Trotter time per layer at saturation ($\chi = 50$) for square grids on MI100. Time scales linearly with the number of bonds n_{bonds}.

Grid	Sites	Bonds	trotter/layer (ms)	Ratio to 3×3
3×3	9	12	∼3,900	1.0
4×4	16	24	∼13,000	3.3
5×5	25	40	∼24,000	6.2
6×6	36	60	∼41,000	10.5
7×7	49	84	∼56,000	14.4
8×8	64	112	∼81,000	20.8

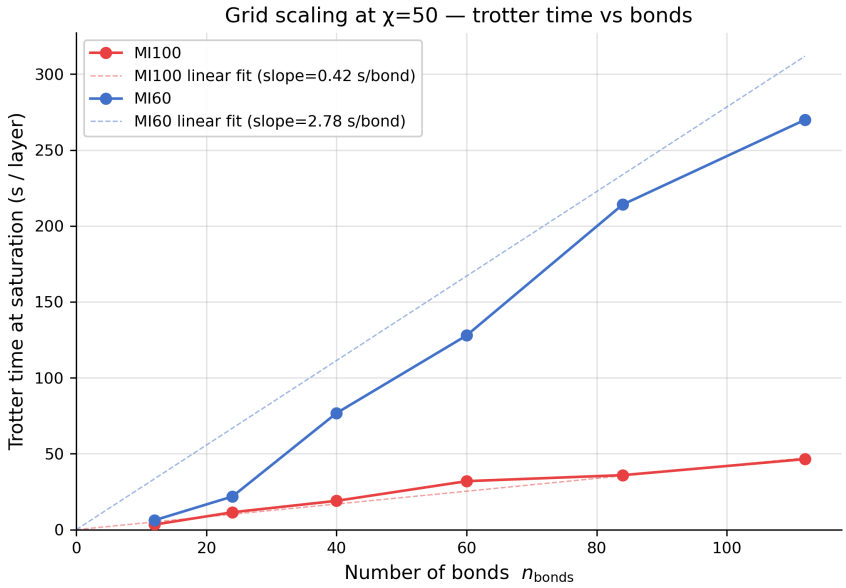


Table 5: Trotter time at $\chi = 50$ saturation vs. number of bonds for square grids 3×3 through 8×8, on MI100 (red) and MI60 (blue). Dashed lines are linear fits through the origin; the legend reports the per-bond slope. MI100 follows the linear model closely (slope 0.42 s/bond), confirming that gate cost is independent of grid size and total layer time is simply proportional to the number of bonds. MI60 departs from its linear fit at larger grids (slope 2.78 s/bond, but with increasing super-linear deviation), consistent with a growing Tensile cache-miss rate as the number of distinct GEMM shapes per layer increases with grid size.

Table 6: Memory model validation: predicted vs. measured VRAM on 4×4 grid (MI100 and MI60, 32 GB HBM2). Prediction uses equations (15)–(17).

$\chi_{\max}$	Predicted (GB)	Measured VRAM	Error
50	0.82	$\approx 3\%$ (1.0 GB)	$< 1\%$
80	5.31	16% (5.1 GB)	$< 1\%$
100	12.94	38% (12.2 GB)	$< 2\%$
120	26.79	79%† (25.3 GB)	$< 6\%^{\dagger\dagger}$

† Measured on MI60; chi grew to 120 at startup allocation.
†† Early measurement before full chi saturation.

Table 7: Predicted peak VRAM (GB) for $n \times n$ grids at selected bond dimensions, from the analytic model (§5). • fits on MI100/MI60 (≤ 32 GB); ○ fits on MI350/MI250X but exceeds MI100; MI300X (192 GB) covers all entries.

Grid	$\chi = 50$	$\chi = 70$	$\chi = 90$	$\chi = 100$
4×4	0.8 •	3.1 •	8.5 •	12.9 •
5×5	1.3 •	5.1 •	13.8 •	21.0 •
6×6	2.0 •	7.8 •	21.2 •	32.3 ○
7×7	2.9 •	11.3 •	30.7 •	46.7 ○
8×8	4.1 •	15.5 •	42.3 ○	64.4 ○
9×9	5.4 •	20.5 •	56.0 ○	85.3 ○
10×10	6.9 •	26.3 •	71.8 ○	109.3 ○†

† Primary research target; MI250X (128 GB) provides comfortable margin.

Table 8: CppSim vs. Julia reference at $\chi = 50$, 4×4, MI100/AMDGPU.

Implementation	trotter/layer	bp/layer	Total/layer
Julia (AMDGPU, reference)	∼20,000 ms	∼8 ms	∼20,008 ms
CppSim (HIP, this work)	∼13,000 ms	∼1,225 ms	∼14,587 ms
Speedup	**1.5**×	—	**1.4**×

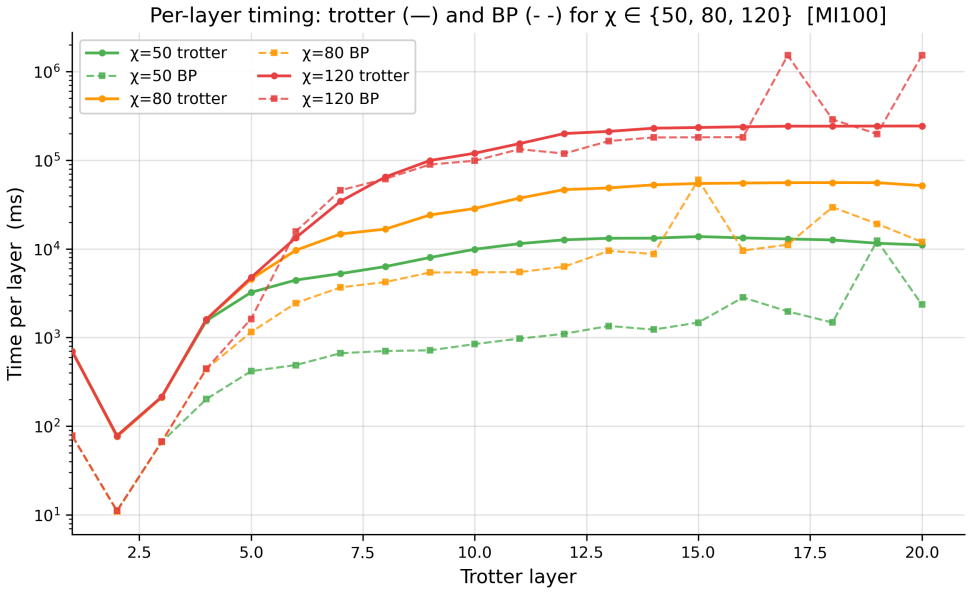


Figure 3: Per-layer timing (ms, log scale) for trotter (solid) and BP (dashed) on MI100, 4×4 grid, for $\chi \in \{50, 80, 120\}$. Three regimes are visible: a ramp-up phase (layers 1–12) where the live bond dimension grows from 1 to $\chi_{\max}$ and cost rises steeply; a saturation plateau where trotter stabilizes at ∼13,000, ∼55,000, and ∼243,000 ms respectively; and BP spikes at hard layers where adaptive convergence requires the full 100-sweep budget. At $\chi = 120$, BP exceeds trotter by up to 5× at layers 16 and 20, making BP the dominant cost at large bond dimension.

Table 9: Hardware comparison: AMD MI100 vs. MI60.

	MI60	MI100
Architecture	GFX906 (Vega)	GFX908 (CDNA1)
FP32 peak	29.5 TFLOPS	184.6 TFLOPS
HBM2 bandwidth	1,024 GB/s	1,229 GB/s
VRAM	32 GB	32 GB
Compute ratio	1×	6.3×
Bandwidth ratio	1×	1.2×

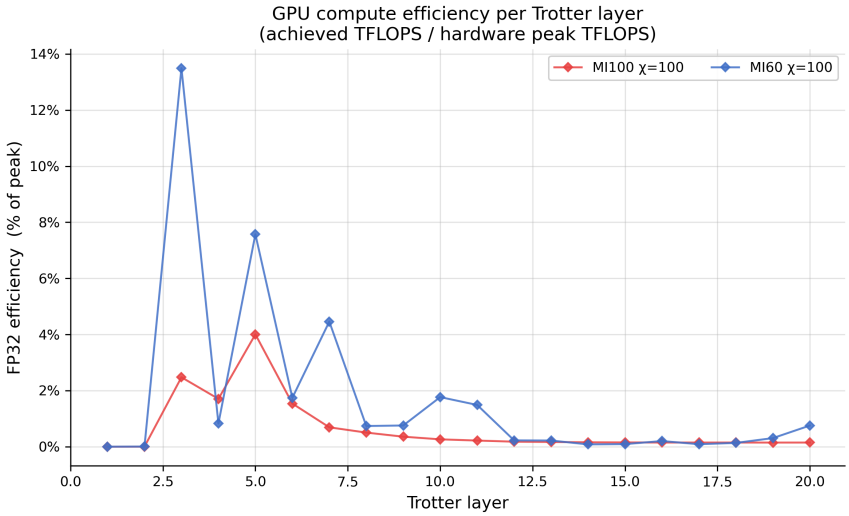


Figure 4: FP32 compute efficiency (achieved TFLOPS / hardware peak TFLOPS, in percent) per Trotter layer for MI100 and MI60 at 4×4, $\chi_{\max} = 100$. Both GPUs peak near $\chi_{\text{live}} \approx 49$ (layer 3), where matrix shapes are large enough for good GPU utilization but have not yet saturated the bond dimension.